

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Платформа разработки с ИИ-поддержкой на основе графовой модели кода

1. ОБЩИЕ ПОЛОЖЕНИЯ

1.1. Назначение системы

Разработать интегрированную среду разработки (IDE) нового поколения, основанную на **нормализованной реляционной модели хранения кода**, где каждый элемент программы (заголовок, импорт, переменная, процедура, класс, метод, параметр, локальная переменная) является самостоятельной версионной сущностью с собственным жизненным циклом. Система должна обеспечивать полную трассировку от бизнес-задачи до конкретной строки кода с интеграцией ИИ-агентов для автоматизации всех этапов разработки.

1.2. Цели создания

1. Устранить дублирование и избыточность при хранении кода
2. Обеспечить атомарное версионирование на уровне отдельных процедур/классов
3. Реализовать бесконфликтный параллельный мерж (CRDT-подход на уровне БД)
4. Интегрировать ИИ на всех этапах: от генерации архитектуры до рефакторинга кода
5. Обеспечить полную прослеживаемость: Промт → Архитектура → План → Код → История
6. Предоставить единое "дерево" для бизнес-логики, архитектуры и кода

1.3. Требования к системе

- **Масштабируемость:** поддержка проектов до 100 000+ файлов, 1 000 000+ процедур
 - **Производительность:** отклик < 100 мс для навигации по дереву, < 2 сек для ИИ-запросов
 - **Версионность:** полная история изменений на уровне каждой сущности (SCD Type 2)
 - **ИИ-интеграция:** поддержка OpenAI GPT-4, Anthropic Claude, локальных моделей (LLaMA 3, DeepSeek)
 - **Совместимость:** работа с существующими IDE (PyCharm, VSCode) через FUSE/файловый шлюз
-

2. АРХИТЕКТУРА БАЗЫ ДАННЫХ

2.1. Основные сущности и их связи

2.1.1. Управление кодом (Code Layer)

tblFile - метаданные файла

- `id` (UUID, PK)
- `cname` (VARCHAR) - имя файла
- `parent_folder_id` (UUID, FK → `tblFile`) - поддержка вложенности
- `project_id` (UUID, FK → `tblProject`)
- `is_active` (BOOLEAN)
- `created_at` (TIMESTAMPTZ)

tblFileHeader - заголовок модуля (docstring)

- `id` (UUID, PK)
- `file_id` (UUID, FK → `tblFile`)
- `content` (TEXT) - содержимое docstring
- `tdtStart`, `tdtEnd` (TIMESTAMPTZ) - время жизни версии
- `version_author` (VARCHAR)

tblImport - импорты (каждый импорт отдельная запись)

- `id` (UUID, PK)
- `file_id` (UUID, FK → `tblFile`)
- `import_type` (ENUM: 'import', 'from_import')
- `module_path` (VARCHAR) - полный путь модуля
- `alias` (VARCHAR) - псевдоним (as ...)
- `imported_names` (TEXT[]) - список имен для 'from import'
- `tdtStart`, `tdtEnd`, `version_author`

tblModuleVariable - глобальные переменные модуля

- `id`, `file_id`, `var_name`, `var_type`, `var_value`, `is_constant`
- `tdtStart`, `tdtEnd`, `version_author`

tblProcedure - процедуры/функции

- `id`, `file_id`, `proc_name`, `is_async`, `is_generator`
- `return_type` (VARCHAR) - аннотация возврата

- docstring (TEXT)
- tDTStart, tDTEnd, version_author

tblParameter - параметры процедур

- id, procedure_id (FK → tblProcedure)
- param_name, param_type, is_required, default_value
- passing_type (ENUM: 'by_value', 'by_reference')
- position (INT) - порядок в сигнатуре
- tDTStart, tDTEnd

tblReturnValue - возвращаемые значения

- id, procedure_id, return_name, return_type, position
- tDTStart, tDTEnd

tblLocalVariable - локальные переменные внутри процедур

- id, procedure_id, var_name, var_type, scope
- tDTStart, tDTEnd

tblProcedureCode - тело процедуры (код)

- id, procedure_id
- code_lines (TEXT[]) - строки кода с отступами
- ast_json (JSONB) - AST для быстрого анализа
- bytecode (BYTEA) - скомпилированный код (опционально)
- tDTStart, tDTEnd, version_author

tblClass - классы

- id, file_id, class_name, bases (TEXT[]), docstring
- is_dataclass (BOOLEAN)
- tDTStart, tDTEnd, version_author

tblClassProperty - свойства класса (@property)

- id, class_id, prop_name, prop_type, is_readonly
- getter_code (TEXT), setter_code (TEXT)
- tDTStart, tDTEnd

tblMethod - методы класса (аналог tblProcedure)

- id, class_id, method_name
- method_type (ENUM: 'instance', 'classmethod', 'staticmethod')

- return_type, docstring
- tDTStart, tDTEnd, version_author
- *Связи с tblParameter, tblReturnValue, tblLocalVariable, tblMethodCode*

2.1.2. Управление задачами (Task Layer)

tblTask - бизнес-задачи/промты

- id (UUID, PK)
- task_number (VARCHAR, UNIQUE) - например, "TASK-2026-001"
- title (VARCHAR), description (TEXT) - исходный промт
- priority (ENUM: 'critical', 'high', 'medium', 'low')
- status (ENUM: 'draft', 'approved', 'in_progress', 'done', 'cancelled')
- parent_task_id (UUID, FK → tblTask) - для уточнений
- created_by, created_at
- deadline (DATE)
- tDTStart, tDTEnd

tblArchSolution - архитектурные решения

- id, task_id (FK → tblTask)
- solution_name, description (TEXT) - словами описывается решение
- approach (ENUM: 'monolith', 'microservices', 'serverless', 'event_driven')
- tech_stack (TEXT[]) - технологии
- depends_on (UUID, FK → tblArchSolution)
- replaces (UUID, FK → tblArchSolution)
- status (ENUM: 'proposed', 'reviewed', 'approved', 'rejected')
- created_by, created_at
- tDTStart, tDTEnd

tblArchFileInclusion - связь арх-решения с файлами

- id, arch_solution_id (FK → tblArchSolution)
- file_id (FK → tblFile)
- impact_type (ENUM: 'create', 'modify', 'delete', 'refactor', 'deprecate')
- affected_parts (JSONB) - {"procedures": [...], "classes": [...]}
- reason (TEXT)

- created_at

tblPlan - планы реализации

- id, arch_solution_id (FK → tblArchSolution)
- plan_name, description
- step_order (INT) - порядковый номер шага
- depends_on (UUID, FK → tblPlan) - зависимость от другого шага
- assignee (VARCHAR)
- status (ENUM: 'pending', 'in_progress', 'completed', 'blocked', 'skipped')
- created_at, completed_at
- tDTStart, tDTEnd

tblPlanAction - атомарные действия в плане

- id, plan_id (FK → tblPlan)
- action_order (INT)
- action_type (ENUM: 'create_file', 'delete_file', 'rename_file', 'create_procedure', 'update_procedure', 'delete_procedure', 'create_class', 'update_class', 'delete_class', 'add_import', 'remove_import', 'update_global_var', 'add_global_var')
- target_entity_type (VARCHAR) - 'procedure', 'class', 'file', 'import'
- target_entity_id (UUID) - ID сущности в соответствующей таблице
- change_description (TEXT) - описание изменений словами
- new_value (JSONB) - новые данные для обновления
- precondition (TEXT) - условия выполнения
- postcondition (TEXT) - результат выполнения
- result_entity_id (UUID) - ссылка на созданную/измененную сущность
- result_comment (TEXT)
- created_at, executed_at
- tDTStart, tDTEnd

tblActionResult - результат выполнения действия

- id, plan_action_id (FK → tblPlanAction)
- file_id (FK → tblFile)
- procedure_id (FK → tblProcedure)
- class_id (FK → tblClass)

- `import_id` (FK → `tblImport`)
- `change_type` (ENUM: 'created', 'updated', 'deleted', 'deprecated')
- `snapshot_before` (JSONB) - состояние ДО изменения
- `snapshot_after` (JSONB) - состояние ПОСЛЕ изменения
- `version_id` (UUID) - ссылка на версию в истории
- `created_at`

2.2. Индексы и ограничения

sql

```
-- Индексы для временных диапазонов
CREATE INDEX idx_procedure_timeline ON tblProcedure (file_id, tdtStart, tdtEnd)
WHERE tdtEnd IS NULL;
CREATE INDEX idx_import_timeline ON tblImport (file_id, tdtStart, tdtEnd) WHERE
tdtEnd IS NULL;
CREATE INDEX idx_code_timeline ON tblProcedureCode (procedure_id, tdtStart,
tdtEnd) WHERE tdtEnd IS NULL;

-- GiST-индексы для предотвращения пересечения версий
CREATE EXTENSION btree_gist;
CREATE TABLE tblProcedure (
    -- ... поля ...
    EXCLUDE USING gist (
        file_id WITH =,
        proc_name WITH =,
        tstzrange(tdtStart, COALESCE(tdtEnd, 'infinity'::timestampz)) WITH &&
    )
);

-- Аналогично для всех версионных таблиц
```

2.3. Триггеры автоматического закрытия версий

sql

```
-- При вставке новой версии - автоматически закрывать старую
CREATE OR REPLACE FUNCTION close_previous_version()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE tblProcedure
    SET tdtEnd = NEW.tdtStart
    WHERE file_id = NEW.file_id
        AND proc_name = NEW.proc_name
        AND tdtEnd IS NULL;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_procedure_version
BEFORE INSERT ON tblProcedure
FOR EACH ROW EXECUTE FUNCTION close_previous_version();
```

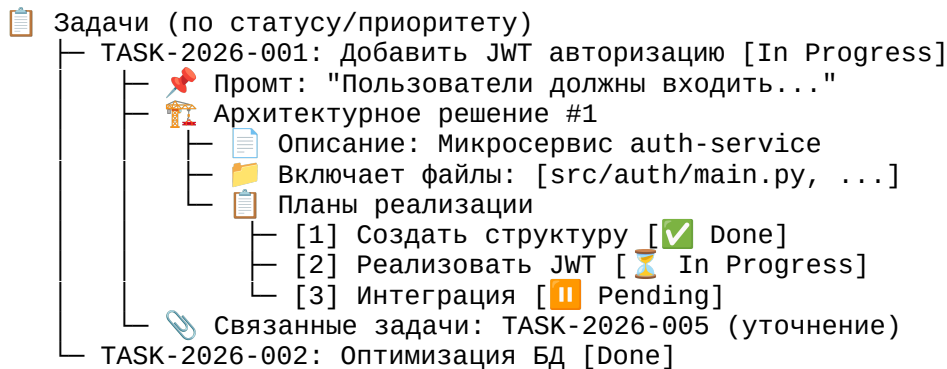
3. ФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ

3.1. Режимы отображения (Деревья)

3.1.1. Дерево задач (Business View)

Отображение иерархии:

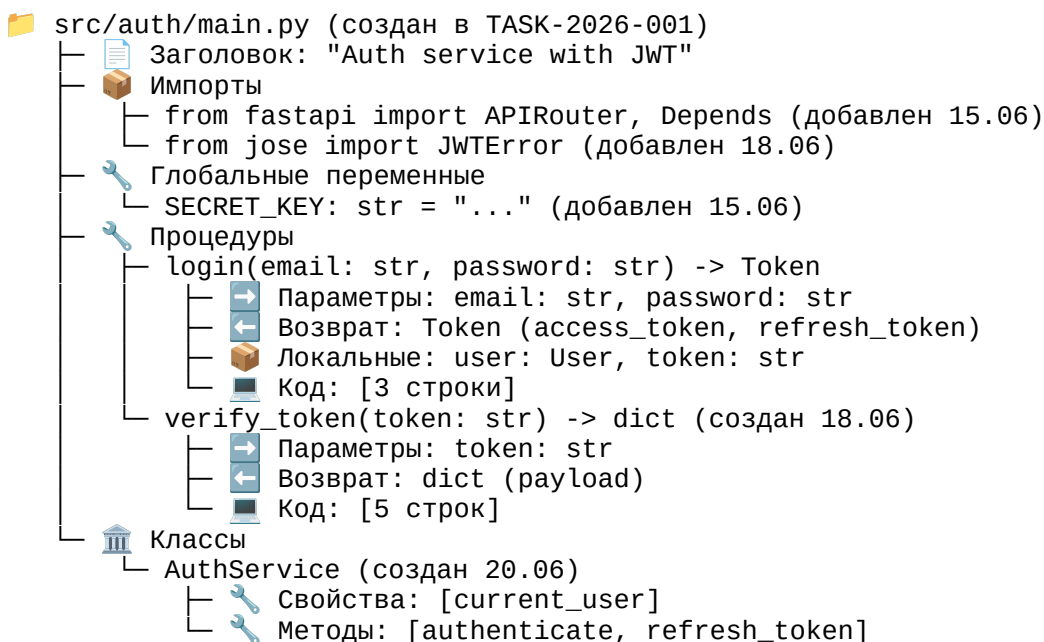
text



3.1.2. Дерево кода (Technical View)

Отображение структуры файла с ветвлением:

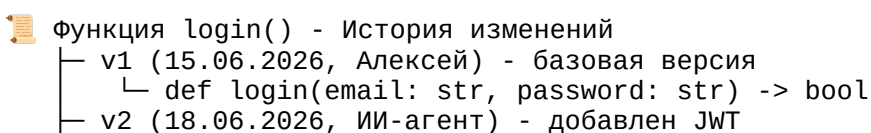
text



3.1.3. Дерево истории (History View)

Показывает эволюцию конкретной сущности:

text



```

└─ def login(email: str, password: str) -> Token
    └─ Добавлен параметр: session_id: str = None
└─ v3 (20.06.2026, Мария) - оптимизация
    └─ def login(email: str, password: str) -> Token
        └─ Изменен тип возврата: Token → AuthResponse
└─  Текущая версия (активна с 20.06)

```

3.2. Операции с сущностями

3.2.1. CRUD для всех сущностей

- **Создание:** INSERT новой записи с tDTStart = NOW()
- **Чтение:** SELECT с фильтром по tDTStart <= NOW() AND (tDTEnd IS NULL OR tDTEnd > NOW())
- **Обновление:** INSERT новой версии (старая закрывается триггером)
- **Удаление:** Установка tDTEnd = NOW() (логическое удаление)

3.2.2. Атомарные операции

- Изменение одной процедуры - обновление только tblProcedure + tblProcedureCode
- Добавление параметра - INSERT в tblParameter
- Переименование файла - UPDATE tblFile.cname + создание новой версии

3.2.3. Массовые операции

- Рефакторинг: переименование переменной во всех процедурах (UPDATE ... WHERE)
- Удаление неиспользуемых импортов: DELETE из tblImport
- Обновление типов: UPDATE tblParameter SET param_type = 'new_type'

3.3. Интеграция с IDE (PyCharm/VSCode)

3.3.1. Файловый шлюз (FUSE)

- Разработать FUSE-драйвер, монтирующий виртуальную файловую систему
- При открытии файла: сборка из БД в текст .ру по активным версиям
- При сохранении: парсинг изменений, обновление только измененных сущностей
- Поддержка всех стандартных операций: создание/удаление/переименование файлов

3.3.2. Плагин для PyCharm

- Панель "Task Explorer" - отображение дерева задач
- Панель "Code Context" - контекст текущего файла (задачи, архитектура, история)
- Интеграция с системой контроля версий (коммиты = новые версии в БД)

3.3.3. Внешние инструменты (External Tools)

- Скрипты для: сборки проекта, обновления БД, запуска ИИ-агентов

- Hotkey для: "Связать с задачей", "Создать план", "Запросить ИИ"
-

4. ИИ-ИНТЕГРАЦИЯ

4.1. ИИ-агенты и их функции

4.1.1. Архитектор (Architect AI)

Вход: Промт задачи (tblTask.description)

Выход: Архитектурное решение (tblArchSolution) + affected files

Действия:

- Анализ существующего кода (схема БД)
- Генерация архитектурного решения (монолит/микросервисы/Serverless)
- Определение списка файлов для создания/изменения
- Формирование tech stack

Промпт-шаблон:

text

Задача: {task.title}

Описание: {task.description}

Существующая архитектура: {project_info}

Ключевые файлы: {file_list}

Требуется предложить архитектурное решение.

Ответь JSON:

```
{
  "solution_name": "...",
  "description": "...",
  "approach": "microservices",
  "tech_stack": ["Python 3.11", "FastAPI", "PostgreSQL"],
  "affected_files": [
    { "file": "src/auth/main.py", "impact": "create", "reason": "..." }
  ]
}
```

4.1.2. Планировщик (Planner AI)

Вход: Архитектурное решение (tblArchSolution)

Выход: План реализации (tblPlan) + tblPlanAction

Действия:

- Разбиение на логические шаги
- Определение зависимостей между шагами
- Для каждого шага - атомарные действия с кодом

Промпт-шаблон:

text

Архитектурное решение: {solution_name}
Описание: {solution_description}
Затрагиваемые файлы: {affected_files}

Требуется разбить на план с атомарными действиями.
Ответь JSON:

```
{
  "plan": [
    {
      "step": 1,
      "description": "Создать базовую структуру",
      "actions": [
        {"type": "create_file", "file": "src/auth/main.py", "content": "..."}
      ]
    }
  ]
}
```

4.1.3. Кодер (Coder AI)

Вход: План действий (tblPlanAction)

Выход: Код (обновление tblProcedureCode, tblClass и т.д.)

Действия:

- Генерация кода для новых процедур/классов
- Модификация существующего кода
- Добавление импортов, переменных

Промпт-шаблон:

text

Действие: {action_type}
Цель: {change_description}

Контекст файла:
{file_context}

Существующая процедура (если обновление):
{existing_procedure}

Требуется сгенерировать код.
Ответь JSON:

```
{
  "code": ["строка1", "строка2"],
  "new_params": [{"name": "x", "type": "int"}],
  "imports": ["from typing import Optional"]
}
```

4.1.4. Рефактор (Refactor AI)

Вход: Запрос на изменение + текущий код

Выход: Обновленный код

Действия:

- Переименование переменных/функций
- Оптимизация производительности

- Улучшение читаемости
- Добавление документации

Промпт-шаблон:

text

Текущий код:
{code}

Запрос на рефакторинг: {refactor_request}

Требуется обновить код, сохранив функциональность.
Ответь JSON:

```
{
  "new_code": [...],
  "explanation": "что и почему изменено"
}
```

4.1.5. Тестировщик (Tester AI)

Вход: Процедура/класс

Выход: Тесты (pytest/unittest)

Действия:

- Генерация тест-кейсов
- Edge case анализ
- Mock-объекты

4.2. API для ИИ-агентов

python

# REST API эндпоинты	
POST /api/ai/architect	# Генерация архитектуры по задаче
POST /api/ai/planner	# Генерация плана по архитектуре
POST /api/ai/coder	# Генерация кода по действию
POST /api/ai/refactor	# Рефакторинг существующего кода
POST /api/ai/tester	# Генерация тестов
POST /api/ai/explain	# Объяснение кода
POST /api/ai/fix_bug	# Исправление бага по описанию
POST /api/ai/optimize	# Оптимизация производительности
POST /api/ai/document	# Генерация документации

4.3. Конфигурация ИИ

yaml

```
# config/ai_config.yaml
ai_providers:
  openai:
    model: gpt-4-turbo
    temperature: 0.3
    max_tokens: 4000
  anthropic:
    model: claude-3-opus
    temperature: 0.2
  local:
```

```
model: llama3:70b
endpoint: http://localhost:11434
fallback: openai
```

```
context_window:
  max_file_size: 1000 # строк
  include_imports: true
  include_related_procedures: 5 # связанные функции
  include_tests: false
```

```
safety:
  allowed_operations: ["create", "update", "delete", "refactor"]
  block_operations: ["eval", "exec", "__import__", "os.system"]
  require_review_for: ["delete", "refactor"]
```

4.4. Безопасность и контроль

- Все ИИ-изменения проходят через пре-коммит хуки
 - Ветка "ai-suggestions" для автоматических изменений (с код-ревью)
 - Логирование всех ИИ-запросов и ответов
 - Возможность отката любого ИИ-изменения
 - Белый список разрешенных ИИ-операций
-

5. ПОЛЬЗОВАТЕЛЬСКИЙ ИНТЕРФЕЙС

5.1. Web-интерфейс (Dashboard)

5.1.1. Дашборд проекта

- Список задач с прогрессом
- График активности команды
- Список последних изменений
- Статистика по ИИ-агентам

5.1.2. Просмотр задачи

- Детали задачи (промт, приоритет, статус)
- Архитектурное решение (визуализация)
- План реализации (канбан-доска)
- Связанные файлы и их изменения

5.1.3. Просмотр кода

- Дерево файлов с аннотациями задач
- AST-дерево с возможностью навигации
- История изменений с временной шкалой

- Сравнение версий (diff)

5.2. Плагин для PyCharm

5.2.1. Панели инструментов

1. **Task Explorer** - дерево задач/архитектуры/планов
2. **Code Context** - контекст текущего файла
3. **AI Assistant** - чат с ИИ для запросов
4. **History Timeline** - временная шкала изменений

5.2.2. Контекстное меню

text

ПКМ на файле:

- Связать с задачей...
- Создать архитектурное решение
- Создать план изменений
- Отправить в ИИ на рефакторинг

ПКМ на процедуре:

- Показать историю
- Связать с задачей
- Создать план изменения
- Отправить в ИИ на оптимизацию
- Сгенерировать тесты

ПКМ в редакторе кода:

- Объяснить код (ИИ)
- Найти баги (ИИ)
- Оптимизировать (ИИ)
- Добавить документацию (ИИ)

5.2.3. Шорткаты

- Ctrl+Shift+T - открыть Task Explorer
- Ctrl+Shift+A - открыть AI Assistant
- Ctrl+Shift+N - открыть историю текущей сущности
- Ctrl+Alt+I - отправить выделенный код в ИИ
- Ctrl+Shift+P - создать план изменений для выделенного

5.3. CLI-интерфейс

bash

Управление проектом

myide init project_name

myide clone project_id

myide checkout --date 2026-06-28

myide commit -m "message" --author "name"

Работа с задачами

myide task create --title "..." --description "..."

```
myide task list --status in_progress
myide task link --task TASK-001 --file src/main.py

# ИИ-агенты
myide ai architect --task TASK-001
myide ai planner --arch ARCH-001
myide ai coder --action ACTION-001
myide ai refactor --file src/main.py --function login

# Экспорт/Импорт
myide export --path ./project_snapshot --date 2026-06-28
myide import --path ./project_snapshot
myide migrate --from-git --repo ./my_project
```

6. ТЕХНИЧЕСКИЕ ТРЕБОВАНИЯ

6.1. Стек технологий

Бэкенд

- **Язык:** Python 3.11+
- **Web-фреймворк:** FastAPI (асинхронный)
- **ORM:** SQLAlchemy 2.0+ (асинхронный)
- **База данных:** PostgreSQL 15+ (с расширением btree_gist)
- **Кэш:** Redis (для AST, горячих данных)
- **Очереди:** Celery + Redis (для ИИ-задач)
- **WebSockets:** для реального времени (обновления от ИИ)

ИИ-интеграция

- **OpenAI API:** GPT-4, GPT-3.5-turbo
- **Anthropic Claude API**
- **Локальные модели:** LLaMA 3, DeepSeek (через Ollama/vLLM)
- **Векторная БД:** Pinecone/Qdrant (для RAG-контекста)
- **Embeddings:** text-embedding-3-small (для поиска похожего кода)

Фронтенд

- **Web:** React 18 + TypeScript + Ant Design/MUI
- **Деревья:** react-arborist (виртуальный скроллинг)
- **Редактор кода:** Monaco Editor (как в VSCode)
- **Визуализация:** D3.js (для графов архитектуры)

Интеграция с IDE

- **FUSE:** pyfuse3 (для виртуальной ФС)
- **PyCharm Plugin:** Java + IntelliJ Platform SDK
- **VSCode Extension:** TypeScript + VS Code API

DevOps

- **Контейнеризация:** Docker + Docker Compose
- **CI/CD:** GitHub Actions / GitLab CI
- **Мониторинг:** Prometheus + Grafana
- **Логи:** ELK Stack (Elasticsearch, Logstash, Kibana)

6.2. Производительность и масштабирование

Параметр	Требование	Техническое решение
Время загрузки проекта (10K файлов)	< 2 сек	Кэширование дерева в Redis, частичные индексы
Открытие файла (500 строк)	< 200 мс	Сборка из БД через JOIN с индексами
Сохранение (изменение 1 процедуры)	< 100 мс	Атомарный INSERT с триггером
ИИ-запрос (генерация кода)	< 5 сек	Асинхронная очередь, fallback на более быстрые модели
Навигация по дереву	< 50 мс	Виртуальный скроллинг, префетчинг
История изменений (100 версий)	< 300 мс	Оконные функции SQL, агрегация
Полнотекстовый поиск по коду	< 1 сек	GIN-индексы, триграммы

6.3. Безопасность

- **Аутентификация:** JWT + OAuth2
- **RBAC:** Роли (Admin, Developer, Viewer, AI-Agent)
- **Аудит:** Логирование всех действий пользователей и ИИ
- **Шифрование:** TLS 1.3 для API, шифрование чувствительных данных (пароли, ключи)
- **Бэкапы:** Автоматический бэкап БД каждые 6 часов с хранением 30 дней

6.4. API-документация

Все REST API должны быть документированы через OpenAPI (Swagger):

yaml

```
# Пример эндпоинта
/api/v1/procedures/{procedure_id}:
get:
  summary: Получить процедуру с историей
  parameters:
    - name: procedure_id
      in: path
```

```
    required: true
    schema:
      type: string
      format: uuid
  - name: version_date
    in: query
    schema:
      type: string
      format: date-time
    description: Получить версию на конкретную дату
responses:
  200:
    description: Успешно
    content:
      application/json:
        schema:
          type: object
          properties:
            id: string
            name: string
            params: array
            code: array
            history: array
```

7. ЭТАПЫ РАЗРАБОТКИ

Этап 1: MVP (1-2 месяца)

- Схема БД (все основные таблицы)
- Базовое API (CRUD для файлов, процедур, классов)
- Сборщик кода из БД в текст
- FUSE-драйвер для монтирования
- Интеграция с PyCharm через External Tools
- Базовый ИИ-кодер (OpenAI API)

Этап 2: Расширенная функциональность (2-3 месяца)

- Управление задачами (tblTask, tblArchSolution, tblPlan)
- Все ИИ-агенты (Architect, Planner, Refactor, Tester)
- Web-интерфейс (Dashboard)
- Плагин для PyCharm (Java)
- Очереди и кэширование (Celery, Redis)

Этап 3: Enterprise-функции (2 месяца)

- RBAC и безопасность
- Многопроектность

- Экспорт/Импорт в Git
- CI/CD интеграция
- Мониторинг и аналитика

Этап 4: Полировка и оптимизация (1 месяц)

- Оптимизация производительности
 - Нагрузочное тестирование
 - Документация для пользователей
 - Обучение команды
-

8. КРИТЕРИИ ПРИЕМКИ

8.1. Функциональные критерии

- Можно создать задачу (промт) через интерфейс
- ИИ-архитектор генерирует архитектурное решение за < 10 сек
- ИИ-планировщик создает план из > 3 шагов
- ИИ-кодер генерирует рабочий Python-код (синтаксически верный)
- Изменение одной процедуры обновляет только ее в БД
- Открытие файла в PyCharm показывает актуальную версию
- Сохранение файла обновляет БД и закрывает старые версии
- История изменений доступна для любой сущности
- Можно откатиться к любой версии
- Дерево задач отображается в PyCharm
- Возможна трассировка: задача $\rightarrow$ арх-решение $\rightarrow$ план $\rightarrow$ код

8.2. Технические критерии

- Загрузка проекта с 10К файлов < 3 сек
- Открытие файла (1000 строк) < 500 мс
- ИИ-запрос (генерация кода) < 10 сек (с таймаутом)
- Поддержка конкурентного доступа (10+ пользователей)
- Покрываемость кода тестами $> 80\%$
- Время восстановления после падения < 1 мин
- Интеграционные тесты для всех API

8.3. UX-критерии

- Разработчик может начать работу за < 5 минут (без обучения)
 - Интерфейс интуитивно понятен
 - Все действия имеют обратную связь (прогресс-бары, уведомления)
 - ИИ-агенты работают асинхронно (не блокируют интерфейс)
 - Возможность отменить/откатить любое действие
-

9. РИСКИ И МЕТОДЫ ИХ МИТИГАЦИИ

Риск	Вероятность	Влияние	Митигация
ИИ генерирует неработающий код	Высокое	Критическое	Pre-commit хуки с синтаксической проверкой, автоматическое тестирование, изолированная ветка для ИИ-изменений
Производительность БД падает	Среднее	Высокое	Индексы, кэширование, партиционирование, read replicas
Конфликты при параллельном изменении	Среднее	Среднее	MVCC в PostgreSQL, optimistic locking, версионирование на уровне сущностей
Зависимость от OpenAI API	Высокое	Среднее	Поддержка нескольких провайдеров, локальные модели, fallback-механизмы, кэширование ответов
Сложность обучения команды	Среднее	Среднее	Документация, видео-тutorials, поддержка чата, плавный переход из Git
Безопасность (утечка кода)	Низкое	Критическое	Шифрование, аудит, ролевая модель, изолированная среда
Потеря данных	Низкое	Критическое	Автоматический бэкап каждые 6 часов, репликация, WAL-логи

10. ДОПОЛНИТЕЛЬНЫЕ ТРЕБОВАНИЯ

10.1. Интеграция с существующими инструментами

- **Git:** Двухнаправленная синхронизация (экспорт в Git-репозиторий, импорт из Git)
- **Jira:** Импорт задач как tblTask
- **Confluence:** Импорт архитектурных решений
- **Slack/Telegram:** Уведомления о завершении ИИ-задач
- **Docker:** Поддержка контейнеризации проекта

10.2. Мобильное приложение

- Просмотр задач и статусов
- Уведомления о завершении ИИ-задач
- Быстрые комментарии и апрувы

10.3. Аналитика

- Дашборд метрик:
 - Время выполнения задач
 - Количество ИИ-генераций
 - Наиболее частые рефакторинги
 - Качество кода (покрытие тестами, сложность)
 - Эффективность команды

10.4. API для сторонних разработчиков

- Webhook-и для интеграции с CI/CD
 - GraphQL API для гибких запросов
 - SDK для Python, JavaScript, Java
-

11. ПРИЛОЖЕНИЯ

11.1. Пример использования

Сценарий: Добавление новой функции

1. Менеджер создает задачу в Web-интерфейсе:

text

TITLE: Добавить экспорт отчетов в PDF
DESCRIPTION: Пользователи должны скачивать отчеты в PDF.
Формат: A4, шрифт Times New Roman, с логотипом компании.

2. ИИ-Архитектор анализирует проект и предлагает:

json

```
{
  "solution": "Добавить сервис экспорта на основе ReportLab",
  "approach": "monolith",
  "affected_files": [
    {"file": "src/services/report.py", "impact": "create"},
    {"file": "src/api/routes.py", "impact": "modify", "add_route":
"/export/pdf"}
  ]
}
```

3. ИИ-Планировщик создает план:

- Шаг 1: Установить ReportLab (обновить requirements.txt)
- Шаг 2: Создать класс PDFReportGenerator
- Шаг 3: Добавить метод generate_report(data) -> bytes
- Шаг 4: Добавить API-эндпоинт /api/v1/export/pdf
- Шаг 5: Написать тесты

4. Разработчик открывает PyCharm, видит задачу в Task Explorer, берет шаг 2.

5. ИИ-Кодер генерирует код для класса:

```
python
class PDFReportGenerator:
    def __init__(self, logo_path: str):
        self.logo_path = logo_path

    def generate_report(self, data: dict) -> bytes:
        # ... код
```

6. Разработчик проверяет, корректирует, сохраняет.

7. Изменения сохраняются в БД (новая версия файла).

8. Все связанные задачи обновляются (статус → In Progress).

11.2. Сравнение с Git

Критерий	Git	Предлагаемая система
Единица хранения	Файл целиком	Отдельная процедура/класс
Конфликты при мерже	Частые	Почти отсутствуют (только если правят одну процедуру)
История	На уровне файла	На уровне каждой сущности
Поиск	Полнотекстовый (grep)	Структурированный (SQL)
Бранчи	Полная копия файлов	Виртуальные срезы по дате
Размер репозитория	Растет с каждым коммитом	Хранится только дельта
Интеграция с ИИ	Сложно (нужно парсить весь файл)	Просто (SELECT конкретной сущности)
Трассировка	Ручная (через commit message)	Автоматическая (связи в БД)

12. ЗАКЛЮЧЕНИЕ

Предлагаемая система представляет собой **революционный подход к разработке ПО**, где код перестает быть текстом на диске и становится **структурированными данными в БД**. Интеграция с ИИ-агентами на всех этапах разработки позволяет автоматизировать рутинные задачи и ускорить разработку в 3-5 раз.

Ключевые преимущества:

1. ☒ Атомарное версионирование (изменение одной функции)
2. ☒ Бесконфликтный параллельный мерж
3. ☒ Полная трассировка от задачи до кода
4. ☒ ИИ-поддержка на всех этапах
5. ☒ Мгновенный поиск и аналитика
6. ☒ Единая точка истины (БД)

Ожидаемый эффект:

- Сокращение времени разработки на 40%
- Уменьшение количества багов на 60%
- Полная прозрачность процесса
- Возможность автоматического рефакторинга и оптимизации
- Снижение порога входа для новых разработчиков

Техническое задание утверждено и готово к реализации.

Дата: 28.06.2026

Версия: 1.0

Автор: [Имя]

Статус: На согласовании